

中英听写_开放平台 AIKit SDK 集成文档

更新记录（能力id：ee62fa27c）

版本	更新日期	更新说明
1.0.10	2024-08-16 15:56	新增君正x2000平台
1.0.1	2023-02-27 14:16	更新安卓32位库
1.0.0	2023-02-23 15:40	当前版本仅支持安卓平台

Android SDK

1. 中英听写_开放平台能力简介

中英听写能力，支持中英和多地方言，支持热词、数字规整等功能

支持多实例并发：否

协议类型：同步流式

2. 授权说明

授权方式支持【设备授权】和【应用授权】2种。

设备授权：按照设备数和有效期授权，激活设备数达到授权量上限后，新设备将无法继续激活使用。SDK采集多个设备标识按照权重算法生成设备指纹精准标识设备，计量准确。支持所有平台。

应用授权：对指定应用授权，仅可在授权的应用上使用，无数量限制，可限制有效期。需提供应用唯一标识，授权能力后，应用级授权支持Android、iOS平台应用。

能力激活支持【在线激活】和【离线激活】2种方式。在能力首次使用时，需要先激活后方可使用。激活时会获取授权license缓存到设备内部存储中。

在线激活：在首次使用时，需要将设备联网，SDK初始化时获取授权license激活。设备激活后，即可在无网环境下使用。如果有恢复出厂设置或清空应用缓存等操作，将license清除后，能力将无法正常使用，将设备联网重启应用即可恢复。适用于设备可联网场景，激活过程简单。

离线激活：将申请的批量激活license文件内置到设备中，在初始化时指定离线激活文件路径，SDK初始化时会自动读取解析本地离线激活文件激活设备，设备激活后，离线激活文件会自动删除。如果有恢复出厂设置或清空应用缓存等操作，将license清除后，能力将无法正常使用，需通过离线激活文件重新激活。适用于设备不可联网或无网场景。

3. 兼容机型

类别	兼容范围
系统	支持Android 5.0 ~ Android 13 版本，鸿蒙系统未做系统兼容性验证
机型	上市的Android手机和平板、及符合具体能力性能要求的Android系统扫描笔、手表等设备
网络	对网络无要求，设备具备联网条件，可使用在线激活方式，首次使用需要连接网络。若设备不能联网，需要使用离线激活方式
开发环境	建议使用 Android Studio 进行开发

4. SDK包组成

DEMO 中已经集成了SDK，您可以参考DEMO，集成SDK。集成前，请先测通DEMO，了解调用原理。如果您自己代码过于复杂，可以使用一个helloworld项目了解集成过程。

将SDK zip包解压缩，得到如下文件：

- Demo //能力SDK Demo、SDK使用说明readme.txt，示例能力调用
- SDK //能力SDK，导入SDK库时使用
- resource //能力对应模型资源，多能力组合时，resource文件夹中包含多个子文件夹
- 测试报告 //SDK 测试报告

- ReleaseNotes.txt //SDK版本日志

5. 接口调用流程



6. 快速集成指南

6.1 导入SDK库

复制 AIKit.aar 到项目的 libs 目录下，然后在项目的 build.gradle 文件中，增加如下配置：

```
dependencies {
    .....// 已忽略无关代码
    implementation fileTree(include: ['*.jar'], dir: 'libs')
    implementation files('libs/AIKit.aar')
}
```

6.2 配置权限

AIKit SDK中使用了如下权限：

权限	使用说明
INTERNET	SDK需要访问网络获取、更新授权
READ_PHONE_STATE	获取设备IMEI用于精准授权，IMEI加密存储，不会存储明文
WRITE_EXTERNAL_STORAGE	SDK写本地日志需要用到该权限
READ_EXTERNAL_STORAGE	SDK加载本地模型资源文件需要该权限
MOUNT_UNMOUNT_FILESYSTEMS	Android 11以上需要该权限
READ_PRIVILEGED_PHONE_STATE	Android 10 以后新增权限，同 READ_PHONE_STATE
MANAGE_EXTERNAL_STORAGE	获取所有文件的访问权限（Android 11以上需要该权限）
READ_PHONE_NUMBERS	获取设备IMEI用于精准授权，IMEI加密存储，不会存储明文（Android 11及以上获取）

SDK中已配置如下权限，如部分权限不需要，可通过如下配置去除

```
<!-- 移除SDK非必须权限示例 -->
<uses-permission android:name="android.permission.READ_PHONE_STATE" tools:node="remove" />
<uses-permission android:name="android.permission.READ_PRIVILEGED_PHONE_STATE" tools:node="remove" />
```

Android 11（API 30）及以上版本需要增加以下权限

```
<uses-permission android:name="android.permission.MANAGE_EXTERNAL_STORAGE" />
<uses-permission android:name="android.permission.READ_PHONE_NUMBERS" />
```

Android 10.0（API 29）及以上版本需要在application中做如下配置

```
<application android:requestLegacyExternalStorage="true"/>
```

6.3 资源导入

复制resource文件夹中文资源到应用的工作目录，即为SDK初始化中的workDir。

6.4 SDK初始化

在使用能力前，需要首先初始化SDK，使用SDK提供的单能力或组合能力时，SDK均只需要初始化一次。

```
// 初始化参数构建
AiHelper.Params params = AiHelper.Params.builder()
    .appId("$appId")
    .apiKey("$apiKey")
    .apiSecret("$apiSecret")
    .workDir("/sdcard/iflytek/aikit")//SDK工作路径，这里为绝对路径，此处仅为示例
    .licenseFile("/sdcard/iflytek/aikit/aee.license")//如果使用离线激活，如要此行配置，指定离线激活文件绝对路径
    .authInterval(60*60*24) //授权更新间隔，这里举例为1天更新一次
    .build();

// 初始化
AiHelper.getInst().initEntry(getApplicationContext(), params);
```

如上代码所示，SDK初始化参数中appId、apiKey、apiSecret 和workDir为必填项。

以下是这些初始化参数：

参数	类型	必填	说明
appId	String	是	应用ID
apiKey	String	是	离线引擎托管平台创建应用后，生成的唯一应用标识
apiSecret	String	是	离线引擎托管平台创建应用后，生成的唯一应用秘钥
workDir	String	是	SDK工作目录
licenseFile	String	否	离线授权license文件绝对路径，SDK离线激活方式，需要从离线引擎托管平台申请的离线授权license文件，放入设备指定路径中（可放入至SDK工作路径下），即licenseFile参数即为离线激活文件的绝对工作路径。
customDeviceId	String	否	用户自定义设备指纹块，默认为空，设置后会成为设备指纹的一部分，建议设置长度低于256
authType	int	否	离线授权类型（0或1），0-->(默认)设备级授权（DEVICE）和 1-->应用级授权（APP）
authInterval	int	否	在线授权校验间隔时长，默认为300s，可自定义设置，最短为60s，单位秒
resDir	String	否	指定资源读取路径，不设置默认从workDir读取
ability	String	否	SDK v2.2.13+版本新增接口，传入需要用到的一个或多个能力ID，格式为“xxx1;xxx2”，多个能力ID中间用 英文分号 隔开，如果注册的能力非空，SDK将对传入的能力ID授权

6.5 注册监听代理

SDK 初始化状态及能力结果可通过registerListener注册监听，在回调方法中获取，SDK的状态和结果回调只支持一个监听器，多次设置会覆盖。可为每个能力指定一个结果回调监听。注册方法及回调接口说明如下：

```
AiHelper.getInst().registerListener(coreListener);// 注册SDK 初始化状态监听
AiHelper.getInst().registerListener("ee62fa27c", aiRespListener);// 注册能力结果监听
```

6.5.1 授权状态回调

授权状态回调 AuthListener 实现示例如下：

```
private AuthListener coreListener = new AuthListener() {
    @Override
    public void onAuthStateChange(final ErrType type, final int code) {
        Log.i(TAG, "core listener code:" + code);
        switch (type) {
            case AUTH:
                Log.i(TAG, "SDK状态: 授权结果码" + code);
                break;
            case HTTP:
                Log.i(TAG, "SDK状态: HTTP认证结果" + code);
                break;
            default:
                Log.i(TAG, "SDK状态: 其他错误");
        }
    }
};
```

SDK仅需要初始化一次，首次初始化成功后，再次初始化不会调用该方法。若初始化失败，再次调用初始化，会再次回调。

- onAuthStateChange:

参数	类型	说明
type	ErrType	SDK错误类型，0 表示授权错误，1 表示 http 请求错误
code	int	错误码，0 表示正常

6.5.2 能力计算进度、结果回调

能力结果及计算进度状态回调示例代码如下：

```
/**
 *能力监听回调
 */
private AiListener aiRespListener = new AiListener() {
    @Override
    public void onResult(int handleID, List<AiResponse> outputData, Object usrContext) {
        if (null != outputData && outputData.size() > 0) {
            for (int i = 0; i < outputData.size(); i++) {
                byte[] bytes = outputData.get(i).getValue();
                if (bytes == null) {
                    continue;
                }
                String key = outputData.get(i).getKey();
                //获取到结果的key及value，可根据业务保存存储结果或其他处理
            }
        }
    }
    @Override
    public void onEvent(int handleID, int event, List<AiResponse> eventData, Object usrContext){
    }
    @Override
    public void onError(int handleID, int err, String msg, Object usrContext){
    }
};
```

- onResult: 能力执行结果返回

参数	类型	说明
handleID	int	会话ID

参数	类型	说明
outputData	List<AiResponse>	能力执行结果
usrContext	Object	用户自定义标识

- onEvent：事件回调

参数	类型	说明
handleID	int	会话ID
event	int	事件状态，0：未知错误；1：开始；2：结束；3：超时；4：进行中
eventData	List<AiResponse>	事件消息数据
usrContext	Object	用户自定义标识

- onError：错误通知，能力执行终止

参数	类型	说明
handleID	int	会话ID
err	int	错误码
msg	String	错误相关标识
usrContext	Object	用户自定义标识

- 能力输出结果 AiResponse 说明：

参数	类型	说明
key	String	输出数据名称
type	enum	输出数据类型，0=音频；1=文本；2=图片；3=视频；5=个性化数据
value	byte[]	字节数组类型输出数据
varType	enum	输出数据参数类型，0=字节数组；1=整型；2=实型；3=布尔型
len	int	输出数据长度

6.6 引擎初始化

当前能力无需调用

6.7 个性化数据接口

个性化数据一般为用户自定义配置文件，用户根据业务需求配置个性化数据，SDK 加载个性化数据示例代码如下：

```
int ret = 0;
AiInput.Builder customBuilder = AiInput.builder();
/**
 * key 数据标识
 * value 数据内容
 * index 数据索引,用户可自定义设置
 */
customBuilder.customText("PERSONAL", "/sdcard/iflytek/aikit/customFile.txt", 0);
customBuilder.customText("PPROC_REPLACE", "/sdcard/iflytek/aikit/customFile.txt", 0);
customBuilder.customText("PPROC_NOT_REP", "/sdcard/iflytek/aikit/customFile.txt", 0);

ret = AiHelper.getInst().loadData("ee62fa27c", customBuilder.build());
if (ret != 0) {
    Log.e(TAG, "loadData 失败: " + ret);
}
//指定要使用的个性化数据集, 未调用, 则默认使用所有loadData加载的数据

int[] indexs = {0};
/**
 * indexs 个性化数据索引数组
 */
AiHelper.getInst().specifyDataSet("ee62fa27c", "PERSONAL", indexs);
AiHelper.getInst().specifyDataSet("ee62fa27c", "PPROC_REPLACE", indexs);
AiHelper.getInst().specifyDataSet("ee62fa27c", "PPROC_NOT_REP", indexs);
if (ret != 0) {
    Log.e(TAG, "specifyDataSet 失败: " + ret);
}
/**
 * 个性化数据预处理
 * key 数据标识
 * value 要生成的个性化资源文件路径
 * index 数据索引
 */
customBuilderPre.customText("PERSONAL", "/sdcard/iflytek/aikit/keyword1.txt" + ".bin", 0);
customBuilderPre.customText("PPROC_REPLACE", "/sdcard/iflytek/aikit/keyword1.txt" + ".bin", 0);
customBuilderPre.customText("PPROC_NOT_REP", "/sdcard/iflytek/aikit/keyword1.txt" + ".bin", 0);

ret = AiHelper.getInst().preProcess("ee62fa27c", customBuilderPre.build());
if (ret != 0) {
    Log.e(TAG, "preProcess 失败: " + ret);
}
```

● 个性化数据参数说明

- 数据段名称: PERSONAL 数据类型: 文本
- 数据段名称: PPROC_REPLACE 数据类型: 文本
- 数据段名称: PPROC_NOT_REP 数据类型: 文本

● loadData 方法参数说明: 设置个性化数据

参数	类型	必填	说明
abilityId	String	是	能力标识ID
request	AiRequest	是	个性化数据构造器

● specifyDataSet 方法参数说明: 指定要使用的个性化数据集, 未调用, 则默认使用所有loadData加载的数据

参数	类型	必填	说明

参数	类型	必填	说明
abilityId	String	是	能力标识ID
key	String	是	个性化资源key
indexs	AiRequest	是	个性化资源索引

- preProcess方法参数说明：为避免个性化数据重复加载耗时，可调用该预处理方法，对原始个性化资源进行预处理，再次调用能力时，则引擎会自 动加载预处理后的资源。

参数	类型	必填	说明
abilityId	String	是	能力标识ID
request	AiRequest	是	个性化资源key

6.8 能力调用接口

6.8.1 创建会话

通过 start 方法开启会话，传入当前能力所需要指定的参数。示例代码如下：

```
AiRequest.Builder paramBuilder = AiRequest.builder();
//paramBuilder.param("lmLoad", false);
//paramBuilder.param("vadLoad", false);
//paramBuilder.param("lmOn", false);
//paramBuilder.param("numLoad", false);
//paramBuilder.param("puncLoad", false);
//paramBuilder.param("vadLinkOn", false);
//paramBuilder.param("vadOn", false);
//paramBuilder.param("postprocOn", false);
//paramBuilder.param("vadResponsetime", 6000);
//paramBuilder.param("vadSpeechEnd", 200);
//paramBuilder.param("dialectType", 0);
//paramBuilder.param("htkNeed", true);
//paramBuilder.param("readableNeed", true);
//paramBuilder.param("pgsNeed", true);
//paramBuilder.param("plainNeed", true);
//paramBuilder.param("vadNeed", true);
//paramBuilder.param("vadEnergyThreshold", 9);
//paramBuilder.param("vadThreshold", 0.1332);
AiHandle handle = AiHelper.getInst().start("ee62fa27c",paramBuilder.build(),null);
if (!handle.isSuccess()) {
    Log.e(TAG, "ERROR::START | handle code:" + handle.getCode());
    return;
}
```

• 功能参数

功能标识	功能描述	数据类型	取值范围	必填	默认值
lmLoad	加载二遍资源	bool	true:加载, false:不加载	否	false
vadLoad	加载VAD资源	bool	true:加载, false:不加载	否	false
lmOn	开启二遍	bool	true:开启, false:不开启	否	false
numLoad	加载数字规整资源	bool	true:加载, false:不加载	否	false
puncLoad	加载标点资源	bool	true:加载, false:不加载	否	false

功能标识	功能描述	数据类型	取值范围	必填	默认值
vadLinkOn	vad子句连接	bool	true:开启, false:不开启	否	false
vadOn	是否开启vad功能	bool	true:开启, false:不开启	否	false
postprocOn	是否开启后处理	bool	true:开启, false:不开启	否	false
vadResponsetime	vad强行结束时间	int	最小值:0, 最大值:999999	否	6000
vadSpeechEnd	vad结束参数	int	最小值:0, 最大值:999999	否	200
dialectType	语言类型	int	最小值:0, 最大值:100	否	0
htkNeed	是否输出htk结果	bool	true:输出, false:不输出	否	true
readableNeed	是否输出readable结果	bool	true:输出, false:不输出	否	true
pgsNeed	是否输出pgs结果	bool	true:输出, false:不输出	否	true
plainNeed	是否输出plain结果	bool	true:输出, false:不输出	否	true
vadNeed	是否输出vad结果	bool	true:输出, false:不输出	否	true
vadEnergyThreshold	vad能量阈值	int	最小值:8, 最大值:16	否	9
vadThreshold	vad模型阈值, 建议0.1332	double	最小值:0.1, 最大值:0.7	否	0.1332

- AiRequest.Builder

参数	类型	必填	说明
key	String	是	功能参数名称
value	String/int/double/bool	是	功能参数取值

- start方法参数说明

参数	类型	必填	说明
ability	String	是	能力标识ID
request	AiRequest	是	能力参数, 可使用AiRequest.Builder快捷构建
usrContext	Object	否	用户自定义标识

返回: AiHandle

AiHandle对象内部提供isSucess方法, 用于判断会话是否启动成功

0=成功, 其他=错误

6.8.2 写入数据

通过调用 write 方法送入能力输入数据。

write方法参数说明

参数	类型	必填	说明
input	AiRequest	是	能力输入, 可使用AiRequest.Builder快捷构建
handle	AiHandle	是	会话的handle对象

返回:

0=成功，其他=错误

送入数据实现代码如下：

```
AiRequest.Builder dataBuilder = AiRequest.builder();
//输入音频数据
byte[] PCM;//byte[]类型音频输入数据
AiAudio.Holder PCMDData = AiAudio
    .get("PCM") //输入数据key
    .encoding(AiAudio.ENCODING_DEFAULT) //设置音频类型
    .data(PCM);//输入数据
PCMDData.status(AiStatus.BEGIN); //送入数据的状态，首帧数据、尾帧数据、中间数据，根据送入数据的状态传入对应的值，取值对应AiStatus.BEGIN、AiStatus.END、AiStatus.CONTINUE
dataBuilder.payload(PCMDData.valid());

int ret = AiHelper.getInst().write(dataBuilder.build(), aiHandle);
//ret 值为0 写入成功；非0失败，请参照文档中错误码部分排查
if (ret != 0) {
    String error = "start write failed" + ret;
    Log.e(TAG, error);
}
```

- 能力输入数据

数据段名称：PCM 数据类型：音频

以下是音频输入参数含义说明：

字段	含义	数据类型	取值范围	默认值	说明	必填
encoding	音频编码	string	lame, speex, opus, speex-wb	speex-wb	取值范围可枚举	否
sample_rate	采样率	int	16000, 8000	16000	音频采样率，可枚举	否
channels	声道数	int	1, 2	1	声道数，可枚举	否
bit_depth	位深	int	16, 8	16	单位bit，可枚举	否
data	音频数据	string			音频大小：0-10M	否
frame_size	帧大小	int	最小值:0，最大值:1024	0	帧大小，默认0	否

6.8.3 获取能力输出结果

送入数据后，需要调用read方法获取到能力结果，read方法为阻塞式方法，当次送入数据处理完毕读取到结果后会返回结果状态码。如果有多次输入，读取到当次结果后，方可进行下一次数据输入。

read方法调用方法如下：

```
try{
    int ret = AiHelper.getInst().read("ee62fa27c", aiHandle);
    if (ret != 0) {
        String error = "start write failed" + ret;
        Log.e(TAG, error);
    }
}
catch (Exception e) {
    e.printStackTrace();
}
```

read方法参数说明：

参数	类型	必填	说明
ability	String	是	能力标识ID
handle	AiHandle	是	会话的handle对象

结果将通过 AIListener 回调返回，AIListener 接口实现可参考 6.5.2 节；能力结果数据格式及含义如下：

- 能力输出数据

数据段名称：output 数据类型：文本

以下是文本输出参数含义说明：

字段	含义	数据类型	取值范围	默认值	说明	必填
encoding	文本编码	string	utf8, gb2312	utf8	取值范围可枚举	否
compress	文本压缩格式	string	raw, gzip	raw	取值范围可枚举	否
format	文本格式	string	plain, json, xml	json	取值范围可枚举	否
data	文本数据	string			文本大小：0-1M	否

6.8.4 结束会话end

```
int ret = AiHelper.getInst().end(aiHandle);
if (ret != 0) {
    String error = "end failed" + ret;
    Log.e(TAG, error);
}
```

参数说明：

参数	类型	必填	说明
handle	AiHandle	是	会话的handle对象

返回：

0=成功，其他=错误

6.9 SDK逆初始化

当不再使用能力时，需调用逆初始化方法释放资源，示例代码如下：

```
AiHelper.getInst().unInit();
```

6.10 导入资源验证

复制resource文件夹下文资源到SDK的工作目录(示例代码工作路径为：” /sdcard/iflytek/aikit “)下，即为SDK初始化中的workDir。资源导入完成后，至此已完成合成能力接入，可运行测试合成效果。

7. 高级功能

7.1 日志配置

```
AiHelper.getInst().setLogInfo(LogLvl.DEBUG, 1, "/sdcard/iflytek/aikit");
```

- 日志配置参数说明

参数	类型	说明
LogLevel	enum	日志级别枚举值如下：DEBUG，INFO，WARN，ERROR，FATAL，OFF
LogMode	int	日志模式（stdout:0 logcat:1 file:2）
LogPath	String	日志路径（路径可根据实际需求进行修改）

7.2 参数配置

可通过 setConfig 方法实现license缓存路径、日志文件上限等配置，使用举例如下：

```
AiHelper.getInstance().setConfig("licenseStoragePath", "/sdcard/iflytek/aikit");
```

- 可配置参数

参数	类型	说明
licenseStoragePath	String	授权文件存储路径, Android平台默认保存在 SharedPreferences下，可通过该配置设置保存在其他路径下。
logMaxSize	String	日志文件上限

7.3 个性化数据卸载

已加载的个性化数据，如果不在使用或者需要替换，需要卸载已加载的个性化资源，示例代码如下：

```
AiHelper.getInstance().unloadData("ee62fa27c", "PERSONAL", indexs);
AiHelper.getInstance().unloadData("ee62fa27c", "PPROC_REPLACE", indexs);
AiHelper.getInstance().unloadData("ee62fa27c", "PPROC_NOT_REP", indexs);
```

- 可配置参数

参数	类型	说明
ability	String	能力id
key_word	String	个性化数据唯一标识
index	String	个性化数据索引

- 个性化数据参数说明

数据段名称：PERSONAL 数据类型：文本

数据段名称：PPROC_REPLACE 数据类型：文本

数据段名称：PPROC_NOT_REP 数据类型：文本

7.4 资源从asset读取

对于模型资源比较小的能力，如果想将模型资源打包到APK或aar 中，可在项目主module 下asset 目录下创建 aikit_resources 目录，将模型资源放置在 aikit_resources文件夹下即可。

Android平台常见问题

- SDK 错误码返回18700，错误日志出现 “online license activate ret:18700 ; auth request failed:2007,未发现SDK” 如何解决？
appid 未授权该SDK 能力，请申请授权后使用该能力。
- SDK 错误码返回18714，错误日志出现 “[{“message”:“HMAC signature cannot be verified: fail to retrieve credential”}]” 如何解决？
appid、apiKey、apiSecret 配置有误，请检查配置。
- SDK 初始化返回 18007，错误日志出现“httpPost failed! 401[{“message”:“HMAC signature does not match”}]; online license activate r

et:18714; AEE_Init ret 18007”，如何解决？

apiKey 或 apiSecret 有误，请检查apiKey、apiSecret 的值是否正确，并替换为正确的值。

4. SDK 初始化时返回18704，错误日志出现“auth request failed:2003,未发现应用”；“AEE_Init ret 18704,”，如何解决？

appid 不存在或appid 未授权，请使用正确的 appid。

5. SDK 初始化时返回18400，错误日志出现“workDir does not have write permission sdcard/xxxx/xxx”，“AEE_Init ret 18400”

SDK 初始化时设置的SDK工作路径即“workDir” 无读写权限，请确保SDK 初始化时，SDK工作路径具有读写权限。

6. Android 平台运行SDK Demo 时，出现如下崩溃，如何解决？

```
Caused by: java.security.InvalidParameterException
at com.iflytek.edgeAI.core.JLibrary$Params$Builder.build(Unknown Source:``41``)
at com.iflytek.aeesdkdemo.sdk.SDKHelper.initSdk(SDKHelper.java:``55``)
at com.iflytek.aeesdkdemo.xtts.XTTSActivity.initSDK(XXXXActivity.java:``144``)
at com.iflytek.aeesdkdemo.xtts.XTTSActivity.onCreate(XXXXActivity.java:``81``)
at android.app.Activity.performCreate(Activity.java:``8214``)
```

崩溃原因为 appid、apiKey、apiSecret 存在为空的值，请正确配置 appid、apiKey、apiSecret。

7. SDK 在初始化引擎时，返回 18601，错误日志出现“AEE_EngineInit:e09712bcb11 ret 18601”；“ability: xxxxxxx not exist”，如何解决？

能力（能力id为 xxxxxx）不存在，初始化引擎时传入的能力id有误，请检查能力id是否正确；或改能力未授权，请确认该能力是否已申请授权。

8. 调用SDK 的start 或 AEE_Start 方法时，返回错误码 18105，错误日志出现“AEE_Start:e09712bcb ret 18105”；“cannot find dependent resource: xxxxxx”，如何解决？

该错误为资源未找到，请检查是否将资源导入到了SDK的工作路径下。

9. 日志中出现“AEE_Start: xxxxxxx ret 18310; AEE_Start failed:18310” 错误日志如何解决？

流式接口能力，调用 AEE_Start 或 start 方法开始的会话，未调用AEE_End 或 end 方法打断或正常结束，再次调用AEE_Srart则SDK会返回18310 错误，同一个能力SDK不支持并发调用，需要结束本次会话之后，才能开始下一次会话。

10. 日志中出现“[ERROR]:[getAbility]-[0474]:ability:64b9b706 not exist AEE_EngineInit: xxxxxxx ret 18601” ， 错误码返回18601 错误，如何解决？

请检查确认引擎初始化及调用能力时所传能力id参数是否正确；如果能力ID无误，SDK日志中打印的 xxxxxx 能力未授权。

11. 日志中出现“auth request failed:2015,无效的指纹”、“online license activate ret:18713”，SDK返回18713错误，如何解决？

应用安全等级与设备指纹不符，须申请调整appid 安全等级；

Harmony SDK

1. 中英听写_开放平台能力简介

中英听写能力，支持中英和多地方言，支持热词、数字规整等功能

支持多实例并发：否

协议类型：同步流式

2. 授权说明

4. SDK包组成

5. 接口调用流程

接口调用流程图

6. 快速集成指南

iOS SDK

1. 中英听写_开放平台能力简介

中英听写能力，支持中英和多地方言，支持热词、数字规整等功能

支持多实例并发：否

协议类型：同步流式

2. 授权说明

授权方式支持【设备授权】和【应用授权】2种。

设备授权： 按照设备数和有效期授权，激活设备数达到授权量上限后，新设备将无法继续激活使用。SDK采集多个设备标识按照权重算法生成设备指纹精准标识设备，计量准确。支持所有平台。

应用授权： 对指定应用授权，仅可在授权的应用上使用，无数量限制，可限制有效期。需提供应用唯一标识，授权能力后，应用级授权支持Android、iOS平台应用。

能力激活支持【在线激活】和【离线激活】2种方式。在能力首次使用时，需要先激活后方可使用。激活时会获取授权license缓存到设备内部存储中。

在线激活：在首次使用时，需要将设备联网，SDK初始化时获取授权license激活。设备激活后，即可在无网环境下使用。如果有恢复出厂设置或清空应用缓存等操作，将license清除后，能力将无法正常使用，将设备联网重启应用即可恢复。适用于设备可联网场景，激活过程简单。

离线激活：将申请的批量激活license文件内置到设备中，在初始化时指定离线激活文件路径，SDK初始化时会自动读取解析本地离线激活文件激活设备，设备激活后，离线激活文件会自动删除。如果有恢复出厂设置或清空应用缓存等操作，将license清除后，能力将无法正常使用，需通过离线激活文件重新激活。适用于设备不可联网或无网场景。

3. 兼容性说明

类别	兼容范围
系统	支持iOS 9.0以上版本
机型	iOS 9.0以上设备
网络	对网络无要求，设备具备联网条件，可使用在线激活方式，首次使用需要连接网络。若设备不能联网，需要使用离线激活方式
开发环境	建议使用 xcode 进行开发

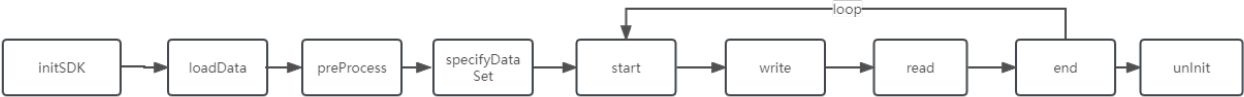
4. SDK包组成

DEMO 中已经集成了SDK， 您可以参考DEMO，集成SDK。集成前，请先测通DEMO，了解调用原理。如果您自己代码过于复杂，可以使用一个helloworld项目了解集成过程。

将SDK zip包解压缩，得到如下文件：

- Demo //能力SDK Demo、SDK使用说明readme.txt，示例能力调用
- SDK //能力SDK，导入SDK库时使用
- resource //能力对应模型资源，多能力组合时，resource文件夹中包含多个子文件夹
- 测试报告 //SDK 测试报告
- ReleaseNotes.txt //SDK版本日志

5. 接口调用流程

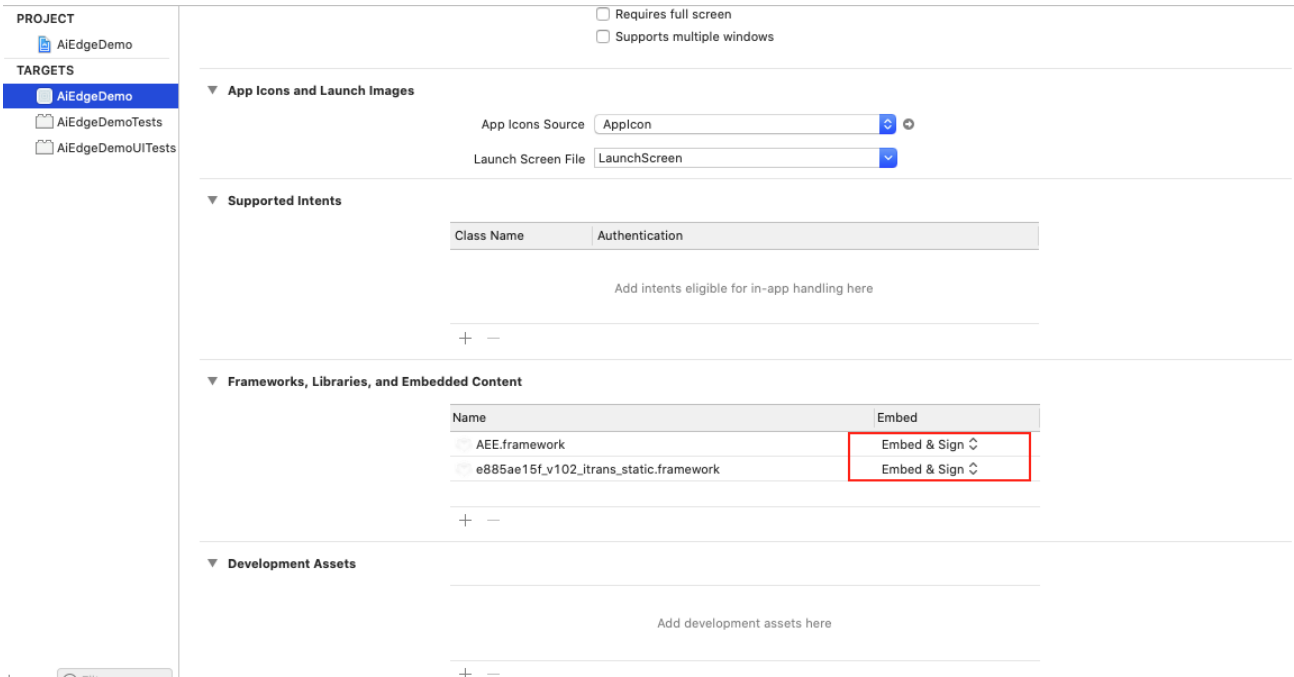


6. 快速集成指南

6.1 导入SDK库

AIKit iOS SDK 核心代码库由AEE.framework 和能力引擎framework 组成。AEE.framework 、引擎framework 均是动态库。在引入工程中时，在工程的General的frameworks,Libraries,and Embedded Content 中使用Embedded & sign。

示例如下

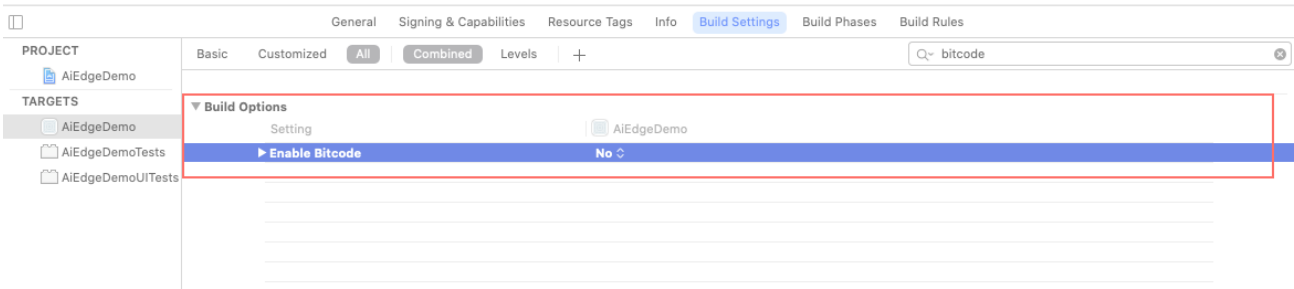


6.2 资源导入

使用 AIKit iOS SDK 提供的能力时，还需要有配套的AI能力模型资源。AIKit 的资源全部存放在AEEResource.bundle下，开发者需将其添加到工程中。

6.3 设置BitCode

AIKit iOS SDK暂不支持Bitcode，Xcode 7、Xcode 8默认开启了Bitcode，而Bitcode 需要工程依赖的所有类库同时支持，可关闭此设置，只需在Targets - Build Settings 中搜索Bitcode 即可，找到相应选项，设置为NO。



6.4 SDK初始化

在使用能力前，需要首先初始化SDK，使用SDK提供的单能力或组合能力时，SDK均只需要初始化一次。

```
// SDK初始化
- (int)initSDK {
    [ILibrary initSDK:^(AiHelperMaker *maker) {
        maker.appId(APPID)
        .apiKey(APIKEY)
        .apiSecret(APISECRET)
        .authInterval(300)
        .logOpen(false)
        .licenseFile("/iflytek/aikit/aee.license"); //非必填参数，离线激活文件路径
    }];
}
```

如上代码所示，SDK初始化参数中appId、apiKey、apiSecret为必填项。

初始化参数通过AiHelperMaker构造，各初始化参数含义如下：

参数	类型	必填	说明
appId	String	是	应用ID

参数	类型	必填	说明
apiKey	String	是	离线引擎托管平台创建应用后，生成的唯一应用标识
apiSecret	String	是	离线引擎托管平台创建应用后，生成的唯一应用秘钥
workDir	String	否	SDK工作目录，默认路径为 “Documents/aikit”
licenseFile	String	否	离线授权license文件绝对路径，SDK离线激活方式，需要从离线引擎托管平台申请的离线授权license文件，放入设备指定路径中（可放入至SDK工作路径下），即licenseFile参数即为离线激活文件的绝对工作路径。
UDID	String	否	用户自定义设备标识，需要服务端配置开启支持后才支持使用，建议设置长度低于256
authType	int	否	离线授权类型（0或1），0-->(默认)设备级授权（DEVICE）和 1-->应用级授权（APP）
resDir	String	否	指定资源读取路径，不设置默认从workDir读取
batchID	String	否	授权批次ID，扩展字段，暂未生效
authInterval	int*	否	授权更新时间，默认最长为1天

6.5 注册回调代理

能力结果可通过registerAbilityCallback 注册回调对象，对象需实现AKitCoreDelegate协议，根据业务需要可为每个能力单独注册回调，或注册一个统一回调。

```
@interface callback : NSObject<AIKitCoreDelegate>
// 构建实现AIKitCoreDelegate协议的callback类
@end

@implementation callback
// 实现数据回调函数
- (void)aikitOnResult:(NSString *)ability
    handleId:(NSInteger)handleId
    outputData:(NSArray<AIKITDataModel *> *)data
    usrCtx:(AIKITCtxContent *)context {
    for (AIKITDataModel *model in data) {
        //获取到结果key 和 reslut
        NSString *key = model.key;
        NSData *value = [[NSData alloc] initWithBytes:model.value length:model.len];
    }
}

// 实现事件回调函数
- (void)aikitOnEvent:(NSString *)ability
    handleId:(NSInteger)handleId
    event:(NSInteger)event
    eventData:(NSArray<AIKITDataModel *> *)data
    usrCtx:(AIKITCtxContent *)context {
    // do something to output event
    if (event == AEE_EVENT_START) {
        // 引擎计算开始
    } else if (event == AEE_EVENT_PROGRESS) {
        // 引擎计算中
    } else if (event == AEE_EVENT_END) {
        // 引擎计算结束
    }
}

// 实现错误信息回调函数
- (void)aikitOnError:(NSString*)ability
    handleId:(NSInteger)handleId
    error:(AIKITError*)error
    usrCtx:(AIKITCtxContent*)context {
    // do something to output error
if (error) {
    NSLog(@"引擎错误，错误类型：%d，错误码：%d，错误描述：%@",error.type, error.code, error.description);
}
}

@end

//支持设置一个全局回调，在方法中通过abilityId区分能力，也支持为每一个能力单独设置回调
callback* cb = [[callback alloc] init];
[AiHelper registerAppCallback:cb]; // global callback：注册为全局级别回调
//[AiHelper registerAbilityCallback:@"ee62fa27c" callback:cb] // ability callback：注册为能力级别回调
```

aikitOnResult参数说明

参数	类型	说明
ability	NSString*	能力标识ID
handleId	NSInteger	能力标识ID
outputData	NSArray<AEED ataModel*>*	能力执行结果
usrCtx	AEEUserConte xt*	用户自定义标识

事件回调 aikitOnEvent参数说明：

参数	类型	说明
ability	NSString*	能力标识ID
handleId	NSInteger	会话id
event	enum	0=未知；1=开始；2=结束；3=超时；4=进度
eventData	NSArray<AEEDataModel*>*	事件消息数据
usrCtx	AIKITCtxContent*	用户自定义标识

aikitOnError: 错误回调

参数	类型	说明
handleId	NSInteger	会话id
usrCtx	AEEUserContext*	用户自定义标识
ability	NSString*	能力标识ID
error	AEEError*	错误消息数据

AEEDataModel对象

参数	类型	说明
key	NSString*	输出数据名称
type	enum	输出数据类型，0=音频；1=文本；2=图片；3=视频；4=个性化数据
value	void*	输出数据
varType	enum	输出数据变量类型，0=字节数组；1=整型；2=实型；3=布尔型
len	int	输出数据长度
status	int	输出数据状态，0=首数据；1=中间数据；2=尾数据；3=非会话单次输入输出

6.6 引擎初始化

当前能力无需调用

6.7 个性化数据接口

个性化数据一般为用户自定义配置文件，用户根据业务需求配置个性化数据，SDK加载个性化数据示例代码如下：

```
AIKITCustomData *customData = [[AIKITCustomData alloc] init];
[customData addTextPath:custom1 key:@"PERSONAL" index:0];
[customData addTextPath:custom1 key:@"PPROC_REPLACE" index:0];
[customData addTextPath:custom1 key:@"PPROC_NOT_REP" index:0];

[AIHelper loadData:ee62fa27c data:customData];
int indexs[] = {0};
[AIHelper specifyDataSet:ee62fa27c key:@"PERSONAL" indexs:indexs count:1];
int indexs[] = {0};
[AIHelper specifyDataSet:ee62fa27c key:@"PPROC_REPLACE" indexs:indexs count:1];
int indexs[] = {0};
[AIHelper specifyDataSet:ee62fa27c key:@"PPROC_NOT_REP" indexs:indexs count:1];

AIKITCustomData *preData = [[AIKITCustomData alloc] init];
[preData addTextPath:$path key:@"PERSONAL" index:indexs];
[preData addTextPath:$path key:@"PPROC_REPLACE" index:indexs];
[preData addTextPath:$path key:@"PPROC_NOT_REP" index:indexs];

ret = [AIHelper preProcess:ee62fa27c data:preData];
```

● 个性化数据参数说明

- 数据段名称: PERSONAL 数据类型: 文本
- 数据段名称: PPROC_REPLACE 数据类型: 文本
- 数据段名称: PPROC_NOT_REP 数据类型: 文本

● loadData 方法参数说明: 指定引擎使用的个性化数据集, 根据能力要求可加载一个或多个。

参数	类型	必填	说明
abilityId	String	是	能力标识ID
customData	AIKITCustomData*	是	个性化数据构造器

● specifyDataSet 方法参数说明: 指定要使用的个性化数据集, 未调用, 则默认使用所有loadData加载的数据

参数	类型	必填	说明
abilityId	String	是	能力标识ID
key	String	是	个性化资源key
indexs	int[]	是	个性化资源索引

● preProcess方法参数说明: 为避免个性化数据重复加载耗时, 可调用该预处理方法, 对原始个性化资源进行预处理, 再次调用能力时, 则引擎会自 动加载预处理后的资源。

参数	类型	必填	说明
abilityId	String	是	能力标识ID
preData	AIKITCustomData*	是	预处理后的个性化资源

6.8 能力调用接口

6.8.1 创建能力会话

通过 start 方法开启会话, 传入当前能力所需要指定的参数。示例代码如下:

```
AIKITParameters *abilityParams = [[AIKITParameters alloc] init];
//[abilityParams addBool:lmLoad value:@"false"];
//[abilityParams addBool:vadLoad value:@"false"];
//[abilityParams addBool:lmOn value:@"false"];
//[abilityParams addBool:numLoad value:@"false"];
//[abilityParams addBool:puncLoad value:@"false"];
//[abilityParams addBool:vadLinkOn value:@"false"];
//[abilityParams addBool:vadOn value:@"false"];
//[abilityParams addBool:postprocOn value:@"false"];
//[abilityParams addInt:vadResponsetime value:6000];
//[abilityParams addInt:vadSpeechEnd value:200];
//[abilityParams addInt:dialectType value:0];
//[abilityParams addBool:htkNeed value:@"true"];
//[abilityParams addBool:readableNeed value:@"true"];
//[abilityParams addBool:pgsNeed value:@"true"];
//[abilityParams addBool:plainNeed value:@"true"];
//[abilityParams addBool:vadNeed value:@"true"];
//[abilityParams addInt:vadEnergyThreshold value:9];
//[abilityParams addDouble:vadThreshold value:0.1332];
AiHandle *handle = [AiHelper start:ee62fa27c param:abilityParams ctxContent:$usrContext];
if (!handle.isSuccess) {
    return;
}
```

功能参数：

功能标识	功能描述	数据类型	取值范围	必填	默认值
lmLoad	加载二遍资源	bool	true:加载，false:不加载	否	false
vadLoad	加载VAD资源	bool	true:加载，false:不加载	否	false
lmOn	开启二遍	bool	true:开启，false:不开启	否	false
numLoad	加载数字规整资源	bool	true:加载，false:不加载	否	false
puncLoad	加载标点资源	bool	true:加载，false:不加载	否	false
vadLinkOn	vad子句连接	bool	true:开启，false:不开启	否	false
vadOn	是否开启vad功能	bool	true:开启，false:不开启	否	false
postprocOn	是否开启后处理	bool	true:开启，false:不开启	否	false
vadResponsetime	vad强行结束时间	int	最小值:0，最大值:999999	否	6000
vadSpeechEnd	vad结束参数	int	最小值:0，最大值:999999	否	200
dialectType	语言类型	int	最小值:0，最大值:100	否	0
htkNeed	是否输出htk结果	bool	true:输出，false:不输出	否	true
readableNeed	是否输出readable结果	bool	true:输出，false:不输出	否	true
pgsNeed	是否输出pgs结果	bool	true:输出，false:不输出	否	true
plainNeed	是否输出plain结果	bool	true:输出，false:不输出	否	true
vadNeed	是否输出vad结果	bool	true:输出，false:不输出	否	true
vadEnergyThreshold	vad能量阈值	int	最小值:8，最大值:16	否	9
vadThreshold	vad模型阈值，建议0.1332	double	最小值:0.1，最大值:0.7	否	0.1332

start 方法说明：

参数	类型	必填	说明
ability	String	是	能力标识ID
abilityParameters	AIKITParameters*	是	能力参数配置，可设置一个或多个
usrContext	AEECtxContent*	否	用户自定义标识

返回：AiHandle

AiHandle对象内部提供isSuccess方法，用于判断会话是否启动成功，通过code属性获取错误码，异常错误请参考文档末尾“常见错误码”章节。

6.8.2 送入数据

通过调用 write 方法送入能力输入数据。

write方法参数说明如下：

参数	类型	必填	说明
input	AIKITInputData*	是	构造能力输入数据及数据描
handle	AiHandle*	是	会话的handle对象

数据构建API参考“7.2节 能力输入数据”说明，送入代码实现数据如下：

```
AIKITInputData *dataBuilder = AIKITInputData.instance();
//输入音频数据
    AIAudio *PCM = AIAudio
        .get("PCM")           //必填项：输入输入音频key
        .data(part)           //必填项：part为 NSData* 类型输入数据
        .status(DataStatusBegin) //非必填：设置输入数据状态，可参考文档7.2节说明及Demo
        .valid();             //必填项：输入数据合法性检查
    dataBuilder.payload(PCM);
int ret = [AiHelper write:dataBuilder handle:$handle];
if (ret != 0) {
    return;
}
```

能力输入数据

数据段名称：PCM 数据类型：音频

以下是音频输入参数含义说明：

字段	含义	数据类型	取值范围	默认值	说明	必填
encoding	音频编码	string	lame, speex, opus, speex-wb	speex-wb	取值范围可枚举	否
sample_rate	采样率	int	16000, 8000	16000	音频采样率，可枚举	否
channels	声道数	int	1, 2	1	声道数，可枚举	否
bit_depth	位深	int	16, 8	16	单位bit，可枚举	否
data	音频数据	string			音频大小：0-10M	否
frame_size	帧大小	int	最小值:0, 最大值:1024	0	帧大小，默认0	否

6.8.3 获取能力输出结果

送入数据后，需要调用read方法读取能力结果，能力结果通过回调代理接口返回，代理实现参考6.5节。read方法为阻塞式方法，当次送入数据处理完毕读取到结果后会返回结果状态码。如果有多次输入，读取到当次结果后，方可进行下一次数据输入。

read方法调用方法如下：

```
ret = [AiHelper read:handle]; // result is output in callback function: $(onResult)
if (ret != 0) {
    NSLog("read error code=%d",ret);
    return;
}
```

read方法说明：

参数	类型	必填	说明
handle	AiHandle*	是	AIKIT_Start生成的会话handle

结果将通过实现AIKitCoreDelegate协议的callback类onResult方法返回，callback 实现可参考6.5节；能力结果数据格式及含义如下：

- 能力结果数据格式及含义如下：

数据段名称：output 数据类型：文本

以下是文本输出参数含义说明：

字段	含义	数据类型	取值范围	默认值	说明	必填
encoding	文本编码	string	utf8, gb2312	utf8	取值范围可枚举	否
compress	文本压缩格式	string	raw, gzip	raw	取值范围可枚举	否
format	文本格式	string	plain, json, xml	json	取值范围可枚举	否
data	文本数据	string			文本大小：0-1M	否

6.8.4 结束会话

数据处理完毕，调用end方法结束会话，示例代码如下：

```
int ret = [AiHelper end:handle];
if (ret != 0)
{
    return;
}
```

end方法说明：

参数	类型	必填	说明
handle	AIKIT_HANDLE*	是	会话的handle对象

6.9 获取能力结果

结果将通过实现AIKitCoreDelegate协议的callback类onResult方法返回，callback 实现可参考 6.5节

6.10 SDK逆初始化

当不再使用能力时，需调用逆初始化方法释放资源，示例代码如下：

```
BOOL ret = [ILibrary unInit];
```

6.11 导入资源验证效果

能力模型资源在AEEResource.bundle下，开发者确保资源已添加到工程中。至此已完成能力接入，可运行测试合成效果。

7、参数构建API说明

7.1 功能参数构建

功能参数通 AEEParameters 类构建，构建方法如下：

- 整型参数构建 [param addInt:PARAM_KEY value:PARAM_VALUE];

参数	类型	必填	说明
key	NSString*	是	功能参数名称
value	NSInteger	是	功能参数取值

- 布尔参数构建 [param addBool:PARAM_KEY value:PARAM_VALUE];

参数	类型	必填	说明
key	NSString*	是	功能参数名称
value	BOOL	是	功能参数取值

- 字符串参数构建 [param addString:PARAM_KEY value:PARAM_VALUE];

参数	类型	必填	说明
key	NSString*	是	功能参数名称
value	NSString*	是	功能参数取值

- 浮点参数构建 [param addDouble:PARAM_KEY value:PARAM_VALUE];

参数	类型	必填	说明
key	NSString*	是	功能参数名称
value	Double	是	功能参数取值

示例代码如下：

```
AIKITParameters *abilityParams = [[AIKITParameters alloc] init];
[abilityParams addString:@"stringKey" value:@"stringValue"];
[abilityParams addDouble:@"doubleKey" value:1.001];
[abilityParams addInt:@"intKey" value:100];
[abilityParams addBool:@"boolKey" value:true];
```

7.2 能力输入数据

输入数据通过AIKITInputData构造，通过 addPayload 方法添加输入数据，输入数据支持文本、图片、音频、视频等格式，分别通过AIText、AIImage、AIAudio、AIVideo封装输入数据及输入数据属性。

- AIAudio 类接口说明

接口	参数说明	方法说明	是否必填
+get:	key: NSS tring*	设置输入数据key	是
-stat us:	status: DataStat us	设置当次输入数据状态，标识当前数据是首次输入、中间数据、结束数据、一次性输入4种类型，对应DataStatusBegin、DataStatusContinue、DataStatusEnd、DataStatusOnce	否
-begi n		快速标识数据首帧状态，等同于aiText.status(DataStatusBegin)	否

接口	参数说明	方法说明	是否必填
-cont		快速标识数据中间数据，等同与aiText.status(AiStatus.DataStatusContinue)	否
-end		快速标识数据尾数据，等同与aiText.status(DataStatusEnd)	否
-once		快速标识数据状态，一次性输入，等同与aiText.status(DataStatusOnce)	否
-data:	data: NSData*	设置输入参数	是
-encoding:	encoding: NSString*	设置音频编码	否
-channels:	channels: NSInteger	设置声道数	否
-bitDepth:	bitDepth: NSInteger	设置位深	否
-sampleRate:	dims:NSInteger	设置采样率	否
-valid		输入数据校验，返回AIText对象	是

示例代码如下：

```
//输入音频数据
AIAudio *aiAudio = AIAudio
    .get("$key")           //必填项：输入输入音频key
    .encoding($encoding)   //非必填：设置音频编码格式
    .sampleRate($sampleRate) //非必填：设置音频采样率
    .channels($channels)   //非必填：设置音频声道数
    .bitDepth($bitDepth)  //非必填：设置音频位深
    .data(part)            //必填项：part为 byte[]类型输入数据
    .status(AiStatus.BEGIN) //非必填：设置输入数据状态
    .valid();              //必填项：输入数据合法性检查
```

8. 高级功能

8.1 日志配置

```
[AiHelper setLogInfo:(AIKITLogLvl)logLevel mode:logMode path:docPath];
```

日志配置参数说明：

参数	类型	说明
LogLvl	int32_t	日志级别枚举值如下:， AIKITLogInfo ， AIKITLogWarn ， AIKITLogError ， AIKITLogFatal ， AIKITLogOff
LogMode	int	日志模式（stadout:0 logcat:1 file:2）
LogPath	String	日志文件名称，默认保存在沙盒路径下 /AIKIT/log

8.2 个性化数据卸载

已加载的个性化数据，如果不在使用或者需要替换，需要卸载已加载的个性化资源，示例代码如下：

```
[AiHelper unloadData:ee62fa27c key:PERSONAL index:$index]
[AiHelper unloadData:ee62fa27c key:PPROC_REPLACE index:$index]
[AiHelper unloadData:ee62fa27c key:PPROC_NOT_REP index:$index]
```

可配置参数

参数	类型	说明
ability	String	能力id
key	String	个性化数据唯一标识
index	String	个性化数据索引

iOS平台常见问题

1. SDK 错误码返回18700，错误日志出现 “online license activate ret:18700 ; auth request failed:2007,未发现SDK” 如何解决？

appid 未授权该SDK 能力，请申请授权后使用该能力。

2. SDK 错误码返回18714，错误日志出现 “[{“message”:“HMAC signature cannot be verified: fail to retrieve credential”}]” 如何解决？

appid、apiKey、apiSecret 配置有误，请检查配置。

3. SDK 初始化返回 18007，错误日志出现“httpPost failed! 401[{“message”:“HMAC signature does not match”}]; online license activate ret:18714; AEE_Init ret 18007”，如何解决？

apiKey 或 apiSecret 有误，请检查apiKey、apiSecret 的值是否正确，并替换为正确的值。

4. SDK 初始化时返回18704，错误日志出现 “auth request failed:2003,未发现应用”；“AEE_Init ret 18704,”，如何解决？

appid 不存在或appid 未授权，请使用正确的 appid。

5. SDK 初始化时返回18400，错误日志出现 “workDir does not have write permission sdcard/xxxx/xxx”，“AEE_Init ret 18400”

SDK 初始化时设置的SDK工作路径即 “workDir” 无读写权限，请确保SDK 初始化时，SDK工作路径具有读写权限。

6. SDK 在初始化引擎时，返回 18601，错误日志出现 “AEE_EngineInit:e09712bcb11 ret 18601”；“ability: xxxxxxx not exist”，如何解决？

能力（能力id为 xxxxxx）不存在，初始化引擎时传入的能力id有误，请检查能力id是否正确；或改能力未授权，请确认该能力是否已申请授权。

7. 调用SDK 的start 或 AEE_Start 方法时，返回错误码 18105，错误日志出现“AEE_Start:e09712bcb ret 18105”；“cannot find dependent resource: xxxxxx”，如何解决？

该错误为资源未找到，请检查是否将资源导入到了SDK的工作路径下。

8. 日志中出现“AEE_Start: xxxxxxx ret 18310; AEE_Start failed:18310” 错误日志如何解决？

流式接口能力，调用 AEE_Start 或 start 方法开始的会话，未调用AEE_End 或 end 方法打断或正常结束，再次调用AEE_Srart则SDK会返回18310 错误，同一个能力SDK不支持并发调用，需要结束本次会话之后，才能开始下一次会话。

9. 日志中出现 “[ERROR]:[getAbility]-[0474]:ability:64b9b706 not exist AEE_EngineInit: xxxxxxx ret 18601” ， 错误码返回18601 错误，如何解决？

请检查确认引擎初始化及调用能力时所传能力id参数是否正确；如果能力ID无误，SDK日志中打印的 xxxxxx 能力未授权。

10. 日志中出现 “auth request failed:2015,无效的指纹”、“online license activate ret:18713”，SDK返回18713错误，如何解决？

应用安全等级与设备指纹不符，须申请调整appid 安全等级；

Linux SDK

1. 中英听写_开放平台能力简介

中英听写能力，支持中英和多地方言，支持热词、数字规整等功能

支持多实例并发：否

协议类型：同步流式

2. 授权说明

授权方式支持【设备授权】

设备授权： 按照设备数和有效期授权，激活设备数达到授权量上限后，新设备将无法继续激活使用。SDK采集多个设备标识按照权重算法生成设备指纹精准标识设备，计量准确。支持所有平台。

能力激活支持【在线激活】和【离线激活】2种方式。在能力首次使用时，需要先激活后方可使用。激活时会获取授权license缓存到设备内部存储中。

在线激活：在首次使用时，需要将设备联网，SDK初始化时获取授权license激活。设备激活后，即可在无网环境下使用。如果有恢复出厂设置或清空应用缓存等操作，将license清除后，能力将无法正常使用，将设备联网重启应用即可恢复。适用于设备可联网场景，激活过程简单。

离线激活：将申请的批量激活license文件内置到设备中，在初始化时指定离线激活文件路径，SDK初始化时会自动读取解析本地离线激活文件激活设备，设备激活后，离线激活文件会自动删除。如果有恢复出厂设置或清空应用缓存等操作，将license清除后，能力将无法正常使用，需通过离线激活文件重新激活。适用于设备不可联网或无网场景。

3. 兼容性说明

类别	兼容范围
gcc/g++	v4.8.5+
接口	Linux C++ 接口
网络	对网络无要求，设备具备联网条件，可使用在线激活方式，首次使用需要连接网络。若设备不能联网，需要使用离线激活方式
开发环境	建议使用 VS Studio 进行开发

4. SDK包组成

DEMO 中已经集成了SDK， 您可以参考DEMO，集成SDK。集成前，请先测通DEMO，了解调用原理。如果您自己代码过于复杂，可以使用一个hellowor1d项目了解集成过程。

将SDK zip包解压缩，得到如下文件：

- Demo //能力SDK Demo、SDK使用说明readme.txt，示例能力调用
- SDK //能力SDK，导入SDK库时使用
- resource //能力对应模型资源，多能力组合时，resource文件夹中包含多个子文件夹
- 测试报告 //SDK 测试报告
- ReleaseNotes.txt //SDK版本日志

5. 接口调用流程



6. 快速集成指南

6.1 导入SDK库

将SDK/libs文件夹（包含libAIKit.so、各能力引擎库）、头文件文件夹include存放到项目中，并在环境变量里添加库路径：

```
#include "../include/aikit_biz_api.h"
#include "../include/aikit_constant.h"
```

6.2 配置权限

SDK工作路径需要读写权限用于存储日志及授权license，缺少读写权限，能力将无法正常使用。

6.3 资源导入

复制resource文件夹中文资源到应用的工作目录，即为SDK初始化中的workDir。该路径需要赋可读、可写的权限。

6.4 SDK初始化

在使用能力前，需要首先初始化SDK，使用SDK提供的单能力或组合能力时，SDK均只需要初始化一次。SDK V2. 2. 13+版本SDK初始化示例代码如下(2. 12及之前版本SDK初始化参考后文)：

```
AIKIT_Configurator::builder()
    .app()
    .appID("")
    .apiKey("")
    .apiSecret("")
    .workDir("")
    .log()
    .logLevel(LOG_LVL_INFO)
    .logPath("./");
int ret = AIKIT_Init();
if(ret != 0){
    printf("AIKIT_Init failed:%d\n",ret);
}
```

SDK初始化参数中appID、apiKey、apiSecret 和workDir为必填项。

初始化配置选项分为四个部分，分别是应用、授权、日志相关配置，可通过链式的调用方式去配置相关参数。

模块	说明	配置选项	类型	必填	说明
app	配置SDK应用相关信息	appID	string	是	应用ID
		apiKey	string	是	离线引擎托管平台创建应用后，生成的唯一应用标识
		apiSecret	string	是	离线引擎托管平台创建应用后，生成的唯一应用密钥
		workDir	string	是	SDK工作目录
		resDir	string	否	指定资源读取路径，不设置默认从workDir读取
		cfgFile	string	否	AIKit配置文件位置，如果为空，则不使用
		authType	string	否	离线授权类型（0或1），0-->(默认)设备级授权（DEVICE）和 1-->应用级授权（APP）
		licenseFile	string	否	离线授权license文件绝对路径，SDK离线激活方式，需要从离线引擎托管平台申请的离线授权license文件，放入设备指定路径中（可放入至SDK工作路径下），即licenseFile参数即为离线激活文件的绝对工作路径。

模块	说明	配置选项	类型	必填	说明
auth	配置SDK授权相关信息	channelID	string	否	渠道ID，存在多个能力集可指定使用的能力集
		UID	string	否	用户自定义设备标识，需要服务端配置开启支持后才支持使用，建议设置长度低于256
		ability	string	否	竞争授权注册的能力id组合，格式如“xxxxx1;xxxxx2”，多个能力通过英文分号隔开，如果进行设置，则按照注册的能力进行授权，如果设置为空或者没有设置，则按照旧版sdkid的方式进行授权
		abilityURL	string	否	配置单个在线能力id及对应请求地址，如需配置多个能力，可多次调用，url格式参考： https://cn-huadong-1.xf-yun.com/v1/private/xxx:443
log	配置SDK日志信息	logLevel	int	否	日志的输出等级： LOG_LVL_DEBUG: debug级别，日志最全 LOG_LVL_WARN: 警告级别信息 LOG_LVL_INFO: 重要信息 LOG_LVL_ERROR: 错误级别日志 LOG_LVL_FATAL: 错误级别日志 LOG_LVL_OFF: 错误级别日志
		logMode	int	否	模式： LOG_STDOUT: linux 控制台 LOG_FILE: 文件形式
		logPath	string	否	输出模式为文件时的文件名称（带有文件名称的路径）

SDK V2. 2. 12 及之前的版本，采用如下方法初始化，2. 2. 13及以后的版本建议使用上文方式初始化，新版本将不在维护老接口。示例如下：

```
AIKIT_InitParam param = {0};
param.appID = "";
param.apiKey = "";
param.apiSecret = "";
param.authType = 0;
param.licenseFile = nullptr; //如果采用离线方式激活方式，需要指定批量离线激活路径
param.workDir = "./";
int ret = AIKIT_Init(&param);
if(ret != 0){
    printf("AIKIT_Init failed:%d\n",ret);
    goto exit;
}
```

如上代码所示，SDK初始化参数中appID、apiKey、apiSecret 和workDir为必填项。

参数说明

参数	类型	必填	说明
appID	String	是	应用ID
apiKey	String	是	离线引擎托管平台创建应用后，生成的唯一应用标识
apiSecret	String	是	离线引擎托管平台创建应用后，生成的唯一应用秘钥
workDir	String	否	SDK工作目录

参数	类型	必填	说明
licenseFile	String	否	离线授权license文件绝对路径，SDK离线激活方式，需要从离线引擎托管平台申请的离线授权license文件，放入设备指定路径中（可放入至SDK工作路径下），即licenseFile参数即为离线激活文件的绝对工作路径。
UDID	String	是	用户自定义设备标识，需要服务端配置开启支持后才支持使用，建议设置长度低于256
authType	int	是	离线授权类型（0或1），0-->(默认)设备级授权（DEVICE）和 1-->应用级授权（APP）
resDir	String	是	指定资源读取路径，不设置默认从workDir读取
batchID	String	是	授权批次ID，扩展字段，暂未生效。

6.5 注册回调监听

当前能力无需调用

6.6 引擎初始化

当前能力无需调用

6.7 个性化数据

个性化数据一般为用户自定义配置文件，用户根据业务需求配置个性化数据，SDK 加载个性化数据示例代码如下：

```
AIKIT_CustomBuilder *customData = AIKIT_CustomBuilder::create();
AEE_CustomData *customData = nullptr
customData->text("PERSONAL", ". /customFile.txt", strlen(". /customData.txt"), 0); //传入个性化数据文件路径
customData->text("key_word", ". /customData.txt.bin", strlen(". /customData.txt.bin"), 0); //指定预处理结果文件路径
customData->text("PPROC_REPLACE", ". /customFile.txt", strlen(". /customData.txt"), 0); //传入个性化数据文件路径
customData->text("key_word", ". /customData.txt.bin", strlen(". /customData.txt.bin"), 0); //指定预处理结果文件路径
customData->text("PPROC_NOT_REP", ". /customFile.txt", strlen(". /customData.txt"), 0); //传入个性化数据文件路径
customData->text("key_word", ". /customData.txt.bin", strlen(". /customData.txt.bin"), 0); //指定预处理结果文件路径
int ret = AIKIT_LoadData("ee62fa27c", AIKIT_Builder::build(customData));
int index0[]={0};
AIKIT_SpecifyDataSet("ee62fa27c", "PERSONAL", index0, 0);
AIKIT_SpecifyDataSet("ee62fa27c", "PPROC_REPLACE", index0, 0);
AIKIT_SpecifyDataSet("ee62fa27c", "PPROC_NOT_REP", index0, 0);
AIKIT_PreProcess("ee62fa27c", AIKIT_Builder::build(customDataSrc, &customData));
```

- 个性化数据参数说明

数据段名称：PERSONAL 数据类型：文本

数据段名称：PPROC_REPLACE 数据类型：文本

数据段名称：PPROC_NOT_REP 数据类型：文本

- AIKIT_LoadData 方法参数说明：设置个性化数据

该能力运行之前，需要调用AIKIT_LoadData方法，加载个性化配置数据。

参数	类型	必填	说明
abilityId	char*	是	能力标识ID
data	AIKIT_CustomData*	是	个性化数据构造器

- AIKIT_SpecifyDataSet 方法参数说明：

指定要使用的个性化数据集，未调用，则默认使用所有loadData加载的数据。

参数	类型	必填	说明
abilityId	char*	是	能力标识ID
key	char*	是	个性化资源key
indexs	int[]	是	个性化资源索引数组
count	int	是	个性化数据索引数组成员个数

• AIKIT_PreProcess 方法参数说明：

为避免个性化数据重复加载耗时，可调用该预处理方法，对原始个性化资源进行预处理，再次调用能力时，则引擎会自动加载预处理后的资源。

参数	类型	必填	说明
abilityId	char*	是	能力标识ID
srcData	AIKIT_CustomData*	是	原始数据输入
data	AIKIT_CustomData**	是	结果数据输出

6.8 能力调用

6.8.1 创建能力会话

通过 start 方法开启会话，传入当前能力所需要指定的参数。示例代码如下：

```
AIKIT_ParamBuilder* paramBuilder = nullptr;
paramBuilder = AIKIT_ParamBuilder::create();
//paramBuilder->param("lmLoad", false);
//paramBuilder->param("vadLoad", false);
//paramBuilder->param("lmOn", false);
//paramBuilder->param("numLoad", false);
//paramBuilder->param("puncLoad", false);
//paramBuilder->param("vadLinkOn", false);
//paramBuilder->param("vadOn", false);
//paramBuilder->param("postprocOn", false);
//paramBuilder->param("vadResponsetime", 6000);
//paramBuilder->param("vadSpeechEnd", 200);
//paramBuilder->param("dialectType", 0);
//paramBuilder->param("htkNeed", true);
//paramBuilder->param("readableNeed", true);
//paramBuilder->param("pgsNeed", true);
//paramBuilder->param("plainNeed", true);
//paramBuilder->param("vadNeed", true);
//paramBuilder->param("vadEnergyThreshold", 9);
//paramBuilder->param("vadThreshold", 0.1332);
ret = AIKIT_Start("ee62fa27c", AIKIT_Builder::build(paramBuilder), nullptr, &handle);
if(ret != 0){
    printf("AIKIT_Start failed:%d\n", ret);
    goto exit;
}
```

功能参数说明：

功能标识	功能描述	数据类型	取值范围	必填	默认值
lmLoad	加载二遍资源	bool	true:加载，false:不加载	否	false

功能标识	功能描述	数据类型	取值范围	必填	默认值
vadLoad	加载VAD资源	bool	true:加载，false:不加载	否	false
lmOn	开启二遍	bool	true:开启，false:不开启	否	false
numLoad	加载数字规整资源	bool	true:加载，false:不加载	否	false
puncLoad	加载标点资源	bool	true:加载，false:不加载	否	false
vadLinkOn	vad子句连接	bool	true:开启，false:不开启	否	false
vadOn	是否开启vad功能	bool	true:开启，false:不开启	否	false
postprocOn	是否开启后处理	bool	true:开启，false:不开启	否	false
vadResponsetime	vad强行结束时间	int	最小值:0，最大值:999999	否	6000
vadSpeechEnd	vad结束参数	int	最小值:0，最大值:999999	否	200
dialectType	语言类型	int	最小值:0，最大值:100	否	0
htkNeed	是否输出htk结果	bool	true:输出，false:不输出	否	true
readableNeed	是否输出readable结果	bool	true:输出，false:不输出	否	true
pgsNeed	是否输出pgs结果	bool	true:输出，false:不输出	否	true
plainNeed	是否输出plain结果	bool	true:输出，false:不输出	否	true
vadNeed	是否输出vad结果	bool	true:输出，false:不输出	否	true
vadEnergyThreshold	vad能量阈值	int	最小值:8，最大值:16	否	9
vadThreshold	vad模型阈值，建议0.1332	double	最小值:0.1，最大值:0.7	否	0.1332

AIKIT_Start 方法说明：

参数	类型	必填	说明
ability	char*	是	能力标识ID
param	AIKIT_BizParam*	是	能力的配置和输入，可使用AIKIT_Builder快捷构建
usrContext	void*	否	用户上下文指针，保存在outHandle中，用以回调中使用
outHandle	AIKIT_HANDLE**（指针的指针）	是	生成的会话句柄

返回：AiHandle

AiHandle对象内部提供isSucess方法，用于判断会话是否启动成功

0=成功，其他=错误

6.8.2 送入数据

通过调用 AIKIT_Write 方法送入能力输入数据。

AIKIT_Write 方法说明：

参数	类型	必填	说明
handle	AIKIT_HANDLE*	是	AIKIT_Start生成的会话handle
input	AIKIT_InputData*	是	输入数据

送入数据实现代码如下：

```
AIKIT_DataBuilder* dataBuilder = AIKIT_DataBuilder::create();
//输入音频数据
char* PCM;
AiAudio* PCMDData = AiAudio::get("PCM")->data(PCM, PCMLen)->valid();
dataBuilder->payload(PCMDData);

int ret = AIKIT_Write(handle, AIKIT_Builder::build(dataBuilder));
if (ret != 0) {
    printf("AIKIT_Write:%d\n", ret);
    goto exit;
}
```

- 能力输入数据

数据段名称：PCM 数据类型：音频

以下是音频输入参数含义说明：

字段	含义	数据类型	取值范围	默认值	说明	必填
encoding	音频编码	string	lame, speex, opus, speex-wb	speex-wb	取值范围可枚举	否
sample_rate	采样率	int	16000, 8000	16000	音频采样率，可枚举	否
channels	声道数	int	1, 2	1	声道数，可枚举	否
bit_depth	位深	int	16, 8	16	单位bit，可枚举	否
data	音频数据	string			音频大小：0-10M	否
frame_size	帧大小	int	最小值:0, 最大值:1024	0	帧大小，默认0	否

6.8.3 获取能力输出结果

送入数据后，需要调用AIKIT_Read方法获取到能力结果，AIKIT_Read方法为阻塞式方法，当次送入数据处理完毕读取到结果后会返回结果状态码。如果有多次输入，读取到当次结果后，方可进行下一次数据输入。

read方法调用方法如下：

```
int ret = AIKIT_Read(handle, &output);
if (ret != 0)
{
    printf("AIKIT_Read:%d\n", ret);
    goto exit;
}
```

AIKIT_Read方法说明：

参数	类型	必填	说明
handle	AIKIT_HANDLE*	是	AIKIT_Start生成的会话handle
output	AIKIT_OutputData** (指针的指针)	是	输出数据

获取能力结果

结果通过outputData参数返回

- 能力结果数据格式及含义如下：

数据段名称：output 数据类型：文本

以下是文本输出参数含义说明：

字段	含义	数据类型	取值范围	默认值	说明	必填
encoding	文本编码	string	utf8, gb2312	utf8	取值范围可枚举	否
compress	文本压缩格式	string	raw, gzip	raw	取值范围可枚举	否
format	文本格式	string	plain, json, xml	json	取值范围可枚举	否
data	文本数据	string			文本大小：0-1M	否

6.8.4 结束会话

数据处理完毕，调用AIKIT_End方法结束会话，示例代码如下：

```
int ret = AIKIT_End(handle);
if (ret != 0)
{
    printf("AIKIT_End failed : %d\n", ret);
}
```

AIKIT_End方法说明：

参数	类型	必填	说明
handle	AIKIT_HANDLE*	是	AIKIT_Start生成的会话handle

6.9 SDK逆初始化

当不再使用能力时，需调用逆初始化方法释放资源，示例代码如下：

```
/**
 * SDK逆初始化函数用以释放SDK所占资源 *
 @return 结果错误码，0=成功 */

AIKITAPI int32_t AIKIT_UnInit();
```

7. 高级功能

7.1 日志配置

可根据需要设置日志输入级别、方式等

```
/**
 * 设置日志级别，模式，以及保存路径
 * @param level 日志级别
 * @param mode 日志输出模式
 * @param path 输出模式为文件时的文件名称
 * @return 错误码 0=成功，其他表示失败
 */
AIKITAPI int32_t AIKIT_SetLogInfo(int32_t level, int32_t mode, const char* path);
```

- 日志配置参数说明

参数	类型	说明

参数	类型	说明
level	int32_t	日志的输出等级： LOG_LVL_DEBUG: debug级别，日志最全 LOG_LVL_WARN: 警告级别信息 LOG_LVL_INFO: 重要信息 LOG_LVL_ERROR: 错误级别日志 LOG_LVL_FATAL: 错误级别日志 LOG_LVL_OFF: 错误级别日志
LogMode	int	模式： LOG_STDOUT:linux 控制台 LOG_FILE: 文件形式
path	const char*	输出模式为文件时的文件名称（带有文件名称的路径

7.2 获取设备ID

通过传入字符类型的指针地址，来获取设备ID。

```
/**
 * 获取设备ID
 * @param deviceID 设备指纹输出字符串
 * @return 结果错误码，0=成功
 */
AIKITAPI int32_t AIKIT_GetDeviceID(const char** deviceID);
```

7.3 个性化数据卸载

已加载的个性化数据，如果不在使用或者需要替换，需要卸载已加载的个性化资源，示例代码如下：

```
/**
 * 个性化数据加载
 * @param ability 能力唯一标识
 * @param key 个性化数据唯一标识
 * @param index 个性化数据索引
 * @return 错误码 0=成功，其他表示失败
 */
AIKITAPI int32_t AIKIT_UnLoadData("ee62fa27c", "PERSONAL", int index);
AIKITAPI int32_t AIKIT_UnLoadData("ee62fa27c", "PPROC_REPLACE", int index);
AIKITAPI int32_t AIKIT_UnLoadData("ee62fa27c", "PPROC_NOT_REP", int index);
```

- 可配置参数

参数	类型	说明
ability	String	能力id
key_word	String	个性化数据唯一标识
index	String	个性化数据索引

7.4 设置授权时间间隔

已加载的个性化数据，如果不在使用或者需要替换，需要卸载已加载的个性化资源，示例代码如下

```

/**
 * 设置授权更新间隔，单位为秒，默认为300秒
 * AIKIT_Init前设置
 * @param interval    间隔秒数
 * @return 结果错误码，0=成功
 */

AIKITAPI int32_t AIKIT_SetAuthCheckInterval(uint32_t interval);

```

7.5 获取SDK版本号

```

/**
 * 获取SDK版本号
 * @return SDK版本号
 */

AIKITAPI const char* AIKIT_GetVersion();

```

Linux平台常见问题

1. SDK 错误码返回18700，错误日志出现 “online license activate ret:18700 ; auth request failed:2007,未发现SDK” 如何解决？
appid 未授权该SDK 能力，请申请授权后使用该能力。
2. SDK 错误码返回18714，错误日志出现 “[{“message”:“HMAC signature cannot be verified: fail to retrieve credential”}]” 如何解决？
appid、apiKey、apiSecret 配置有误，请检查配置。
3. SDK 初始化返回 18007，错误日志出现“httpPost failed! 401{“message”:“HMAC signature does not match”}; online license activate ret:18714; AEE_Init ret 18007”，如何解决？
apiKey 或 apiSecret 有误，请检查apiKey、apiSecret 的值是否正确，并替换为正确的值。
4. SDK 初始化时返回18704，错误日志出现 “auth request failed:2003,未发现应用”； “AEE_Init ret 18704,”，如何解决？
appid 不存在或appid 未授权，请使用正确的 appid。
5. SDK 初始化时返回18400，错误日志出现 “workDir does not have write permission sdcard/xxxx/xxx”， “AEE_Init ret 18400”
SDK 初始化时设置的SDK工作路径即 “workDir” 无读写权限，请确保SDK 初始化时，SDK工作路径具有读写权限。
6. SDK 在初始化引擎时，返回 18601，错误日志出现 “AEE_EngineInit:e09712bcb11 ret 18601”； “ability: xxxxxxx not exist”，如何解决？
能力（能力id为 xxxxxx）不存在，初始化引擎时传入的能力id有误，请检查能力id是否正确；或改能力未授权，请确认该能力是否已申请授权。
7. 调用SDK 的start 或 AEE_Start 方法时，返回错误码 18105，错误日志出现 “AEE_Start:e09712bcb ret 18105”； “cannot find dependent resource: xxxxxx”，如何解决？
该错误为资源未找到，请检查是否将资源导入到了SDK的工作路径下。
8. 日志中出现 “AEE_Start: xxxxxxx ret 18310; AEE_Start failed:18310” 错误日志如何解决？
流式接口能力，调用 AEE_Start 或 start 方法开始的会话，未调用AEE_End 或 end 方法打断或正常结束，再次调用AEE_Start则SDK会返回18310错误，同一个能力SDK不支持并发调用，需要结束本次会话之后，才能开始下一次会话。
9. 日志中出现 “[ERROR]:[getAbility]-[0474]:ability:64b9b706 not exist AEE_EngineInit: xxxxxxx ret 18601” ， 错误码返回18601 错误，如何解决？
请检查确认引擎初始化及调用能力时所传能力id参数是否正确；如果能力ID无误，SDK日志中打印的 xxxxxx 能力未授权。
10. 日志中出现 “auth request failed:2015,无效的指纹”、“online license activate ret:18713”，SDK返回18713错误，如何解决？
应用安全等级与设备指纹不符，须申请调整appid 安全等级；

错误码

错误码	含义	自查指南
18000	本地 license 文件不存在	离线激活方式，请检查离线授权文件路径下，license 文件是否存在；如果不存在，请将申请的离线授权文件放置在对路径下； 在线激活方式下，请检查 workDir 目录下 license 文件是否存在，如果不存在，请将设备联网，重启应用。
18001/ 18002/ 18003/ 18004	授权问题	Linux 平台删除授权文件，将设备联网，重启应用，重新从云端获取授权文件； Android 端清除应用缓存，将设备联网，重启应用，重新从云端获取授权文件； 如仍有问题，请联系项目经理支持协助解决。
18005	授权已过期	能力授权已过期。
18006	授权时间错误，设备时间比标准时间慢 30 分钟以上	设备时间不准确导致，请校正设备时间为标准时间，重启应用。
18007	授权应用不匹配（apiKey、apiSecret）	apiKey、apiSecret 配置有误，请核对项目中配置的 apiKey、apiSecret 。
18008	授权文件激活过期	非永久授权激活文件的有效期为 3天，Linux 平台删除授权文件、Android平台清除应用缓存后重启应用重新激活； 若设备无法联网，请重新申请离线激活文件，在 3 天内激活。
18010	离线授权激活文件指定平台与设备平台不匹配	批量授权激活文件不支持当前设备平台，离线批量授权文件仅只支持申请时指定的特定平台，不可多平台混用。
18011	离线授权激活文件指定架构与设备 CPU 架构不匹配	批量授权激活文件不支持当前设备架构，离线批量授权文件仅只支持申请时制定的特定架构，不支持多架构混用。
18012	离线授权激活文件中包含 license 个数异常	离线批量激活文件异常，请检查设备中使用的离线激活文件与所申请的是否一致。
18013	离线授权激活文件中未找到当前设备	批量离线授权激活文件中不包含改设备，请联网激活或申请离线授权激活文件。
18014	离线授权激活文件中设备指纹安全等级非法	离线授权文件中安全等级错误，请联系平台重新提供。
18015	硬件授权验证失败	-
18016	离线授权激活文件内容非法	批量离线授权文件不合法，请替换正确的批量离线激活文件测试，如仍有问题，请联系项目经理协助解决。
18017	离线授权激活文件中协议头非法	批量离线授权文件不合法，请替换正确的批量离线激活文件测试，如仍有问题，请联系项目经理协助解决
18018	离线授权激活文件中指纹组成项个数为0	未获取到有效的设备标识，例如 Android平台 Androidid 未获取到；Linux平台 MAC等未获取到。
18100/ 18101/ 18102	资源错误	请仔细检查所用资源与平台提供的资源是否一致，可通过MD5对比，请使用提供SDK时附带的成套资源。
18103	资源参数不存在	请根据文档检查能力参数Key、Value 是否合法。
18104	资源路径打开失败	资源路径无读写权限，请确保在 SDK 初始化时以获取到读写权限，或将资源路径设置为有读写权限的路径下。
18105	资源加载失败，workDir 内未找到对应资源	请根据日志提示将所需资源导入到指定路径下。

错误码	含义	自查指南
18106	资源卸载失败	请检查代码是否存在未初始化时调用 SDK uninit 方法，或多次调用 uninit 方法
18200	引擎鉴权失败	应用授权信息与SDK不对应。 1. 请尝试将设备联网后重启应用； 2. 请检查您项目中使用的SDK和平台提供的是否一致；
18201	引擎动态加载失败	请检查日志中打印的引擎库是否已导入到项目中
18202	引擎未初始化	请检查调用能力前是否已初始化引擎及初始化是否成功。
18203	引擎不支持该接口调用	请参照 Demo 仔细检查方法调用是否正确。
18205	引擎重复初始化	请根据文档检查引擎初始化方法是否存在重复调用，引擎初始化全局仅需初始化一次。
18206	引擎参数非法	请根据文档检查引擎初始化参数是否合法
18207	引擎上下文未注册	-
18301	SDK未初始化	在使用能力前请先初始化 SDK，如果有调用 uninit 方法，再次使用能力时需要重新初始化。
18302	SDK初始化失败	请根据CoreListener回调中返回的错误码参考此文档做对应检查。
18303	SDK 已经初始化	重复初始化导致，使用能力时，SDK 只需要初始化一次，请检查 SDK 初始化逻辑是否存在多次初始化。
18304	不合法参数	请参考demo及协议文件仔细检查所传参数是否正确；若确认无误，请联网后重启应用后重试。
18305/ 18306	会话不存在或已销毁	1. 请检查 start 方法调用是否返回成功； 2. 请排查代码是否在调用 end 方法结束会话后又调用了 write 或read 方法。 3. 请确保 end 方法在能力执行完毕后调用，能力执行结果会在 onEvent 中回调。
18307	会话重复终止	该错误一般为重复调用 end 方法导致， 请仔细检查代码是否有重复调用 end 方法。
18310	会话重复开启	start 、end方法需成对调用，请仔细检查代码逻辑是否存在调用start 后未调用end结束会话，又再次调用start 情况出现。
18400	工作目录无写权限	在设置 workDir 时，请确保该工作路径有读写权限。若无法设置读写权限，请修改为有读写权限的工作路径。
18401	未获取到设备标识	没有获取到组成设备指纹所必须的组成项，请确认 mac、AndroidID、SN 等是否能获取到，请根据授权安全等级确保能获取对应数量的设备标识。
18402	文件打开失败	请检查 日志中所打印的文件是否存在， 以及对应路径下是否有读权限。
18404	设备指纹比较失败	设备指纹组成项发生变化导致，获取到的设备指纹与缓存的设备指纹不一致，请将设备联网后重启应用重试。
18500	未找到该参数 key	请参照demo或协议文档仔细检查参数名拼写。
18501	参数范围溢出，不满足约束条件	请根据协议检查调用 SDK 方法时所传参数范围，需要确保所传参数符合协议约束要求。
18502	SDK 初始化参数为空	请根据 SDK 集成文件检查 SDK 初始化代码，确保必填参数有值且合法。
18503	SDK 初始化参数中 appId 为空	appId 为空值，请在 SDK 初始化时传入正确的 appId 值。
18504	SDK 初始化参数中 apiKey为空	apiKey为空值，请在 SDK 初始化时传入正确的 apiKey值。
18505	SDK 初始化参数中 apiSecret 为空	apiSecret 为空值，请在 SDK 初始化时传入正确的 apiSecret 值。

错误码	含义	自查指南
18506	abilityId 参数为空	abilityId 为空值，请检查 引擎初始化方法及调用能力方法时所传的 abilityId 是否正确。
18507	input 参数为空	请检查调用能力方法时 input 参数是否为空。
18508	输入数据参数 Key 不存在	请根据协议文档检查输入参数的 key。
18509	必填参数缺失	请根据文档或demo检查参数漏传项。
18510	输出参数缺失	能力输出参数缺失
18520	不支持的编解码类型	输入的编解码格式不支持
18600	协议中时间戳字段缺失	请将设备联网后重启应用。
18601	协议中未找到该能力 ID	请确认 SDK 初始化时所传 appId、 apiKey、apiSecret 是否正确； 若无误请删除授权文件或清空应用缓存后重启应用； 请确认所传能力 ID 是否正确； 请联系平台协助排查授权是否已过期或授权量已用完。
18602	协议中未找到该资源	请确认 SDK 初始化时所传 appId、 apiKey、apiSecret 是否正确； 请将设备联网后重新应用重试； 请确认是否已申请授权过该资源。
18603	协议中未找到该引擎	请确认 SDK 初始化时所传 appId、 apiKey、apiSecret 是否正确； 请将设备联网后重新应用重试； 请确认项目中使用SDK版本是否正确。。
18606	协议能力接口类型不匹配	请根据协议检查能力接口类型是否一致，流式与非流式接口不可混用。
18700	能力未授权	该SDK未授权，请申请授权。
18701	网路不通	请检查网络连接是否正常。
18702	网关检查不过	检查设备时间是否正确； 请检查 SDK 初始化时所传 apiKey、apiScret 是否正确。
18703	云端响应格式不对	请检查网络是否可以正常访问外网。
18704	应用未注册	请检查appid 是否正确。
18705	应用 ApiKey & ApiSecret 校验失败	请检查 apiKey、apiSecret 是否正确。
18706	引擎不支持的平台架构	您所使用的能力不支持该设备平台架构，请确认申请的SDK支持的平台架构。
18707	授权已过期	如需继续使用请申请授权延期。
18708	无可用授权	1. 该能力不支持当前平台架构的设备； 2. 授权量已满。
18709	未找到该 appId 绑定的能力	请检查 appId 是否正确；请确认是否已购买该能力。
18710	未找到该 appId 绑定的资源	请检查 appId 是否正确；请检查是否已申请受限资源。
18713	设备指纹安全等级不匹配	设备指纹安全等级不匹配，请申请安全等级变更。
18714	appid 错误	该appid未申请过授权，请检查appid是否正确。
18715	未找到该SDKID	该组合能力SDK未授权，请申请授权。

错误码	含义	自查指南
18716	未找到该组合能力集合	该组合能力未创建，请申请授权。
18717	SDK组合能力授权不足	组合能力授权量已用完，需申请追加授权。
18718	无效授权应用签名	应用级授权，应用签名与授权签名不一致。
18719	应用签名不唯一	应用级授权，提供的应用签名有误。
18801	建立链接错误	
18802	结果等待超时	